

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

**«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ
ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ им. В. Г. ШУХОВА»
(БГТУ им. В.Г. Шухова)**

Кафедра программного обеспечения вычислительной техники и автоматизированных
систем

Лабораторная работа №21
по дисциплине: Основы программирования
тема: **«Исключения»**

Выполнил: студент группы ПВ-221
Дорошенко Владимир Юрьевич

Проверили:
ст. преп. Притчин Иван Сергеевич
асс. Черников Сергей Викторович

Белгород 2023 г.

Цель работы: получение навыков работы с исключениями, осознание необходимости данных языковых средств.

Содержание отчета:

- Тема лабораторной работы.
- Цель лабораторной работы.
- Тексты заданий с набранными фрагментами и ответами на необходимые вопросы. Даже если пункт сделан в ознакомительных целях, в процессе набора вы акцентируете на нужных моментах.

Задание 1

- a) Рассмотрим всё же плохой сценарий обработки (вывод сообщения). Самостоятельно наберите пример ниже и опишите его поведение при b равном и неравном нулю.

```
#include <iostream>

int division(int a, int b) {
    if (b != 0) {
        return a / b;
    } else {
        std::cerr << "b must not be equal to 0" << std::endl;
    }
}

int main() {
    int a, b;
    std::cin >> a >> b;

    auto res = division(a, b);

    std::cout << res;

    return 0;
}
```

Наблюдаемое поведение :

При $b \neq 0$ программа выдаёт значение переменной `res`

При $b = 0$ программа выдаёт ошибку, а после неё выдаёт значение переменной `res`(мусор)

Недостатки : значение переменной всё равно выводиться, хотя при делении на ноль должна быть только ошибка

- b) Каким образом функция `main` из прошлого примера поймёт, что что-то в функции `division` пошло не так? Да, вы можете сказать, что это было известно ещё до вызова функции. Однако заранее знать ситуации, при которых могут возникнуть исключительные ситуации не всегда возможно. Функция `division` хотела бы как-нибудь информировать `main` о том, что произошла проблема. Она могла бы делать это при помощи возвращаемого значения:

```
#include <iostream>

bool division(int a, int b, int &res) {
    if (b != 0) {
        res = a / b;
        return true;
    } else {
        return false;
    }
}

int main() {
    int a, b;
    std::cin >> a >> b;
    int res;

    if (division(a, b, res)) {
        std::cout << res;
    } else {
        std::cerr << "b must not be equal to 0" << std::endl;
    }

    return 0;
}
```

Недостатки : задействуются лишние ресурсы на изменение переменной, данный пример работает только в разрезе `if else`

- с) Для сигнализации исключительной ситуации в языке программирования C++ используется ключевое слово `throw`:

```
#include <iostream>

int division(int a, int b) {
    if (b == 0) {
        throw std::string("Division by zero");
    }

    return a / b;
}

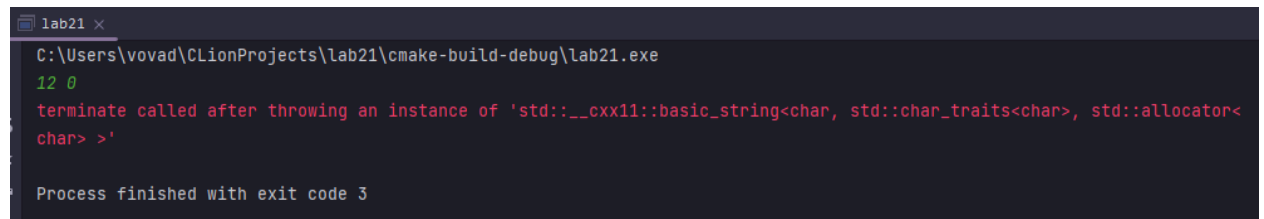
int main() {
    int a, b;
    std::cin >> a >> b;

    int res = division(a, b);

    std::cout << res << std::endl;

    return 0;
}
```

Вывод при `b = 0` :



```
lab21 x
C:\Users\vovad\CLionProjects\lab21\cmake-build-debug\lab21.exe
12 0
terminate called after throwing an instance of 'std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> >'
Process finished with exit code 3
```

- d) Throw прокидывает исключение вызывающей стороне. Считается, что она (вы зывающая сторона) каким-то образом знает, как обработать данную исклю чительную ситуацию на основании полученного исключения. Для перехвата исключения используется конструкция вида:

```
#include <iostream>

int division(int a, int b) {
    if (b == 0) {
        throw std::string("Division by zero");
    }

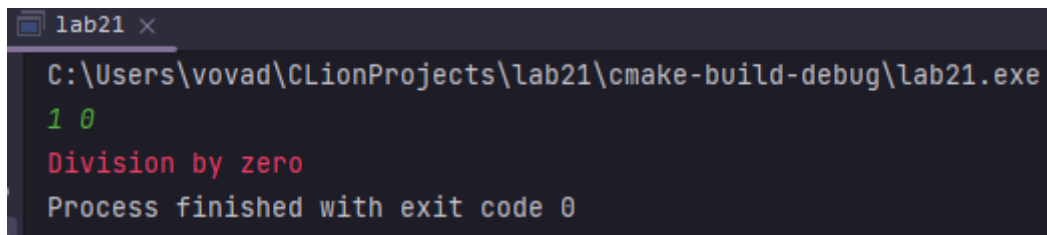
    return a / b;
}

int main() {
    int a, b;
    std::cin >> a >> b;

    try {
        auto res = division(a, b);
        std::cout << res;
    } catch (const std::string &s) {
        std::cerr << s;
    }

    return 0;
}
```

Наблюдаемое поведение :



```
lab21 x
C:\Users\vovad\CLionProjects\lab21\cmake-build-debug\lab21.exe
1 0
Division by zero
Process finished with exit code 0
```

Выдаёт ошибку описанную в функции

- e) На основании описания поведения абзацем выше объясните, каким образом работает данный фрагмент:

```
#include <iostream>

void f1() {
    throw std::string("ERROR!!!");
}

void f2() {
    try {
        f1();
    } catch (int a) {
        std::cerr << "CATCH IN F2";
        std::cerr << "(" << a << ")";
    }
}

int main() {
    try {
        f2();
    } catch (std::string &s) {
        std::cerr << "CATCH IN MAIN";
        std::cerr << "(" << s << ")";
    }

    return 0;
}
```

Объяснение :

Так как тип ошибки из f1() не соответствует типу в catch из f2() она записывается в catch из main

f2()

```
catch (int a)
```

main

```
catch (std::string &s)
```

и перехваченное значение выводиться в консоль

```
C:\Users\vovad\CLionProjects\lab21\cmake-build-debug\lab21.exe
CATCH IN MAIN(ERROR!!!)
Process finished with exit code 0
```

- f) Напишите фрагмент кода, который выбирает необходимый catch в зависимости от типа исключения (конструкцию с несколькими блоками catch)

```
#include <iostream>

void f1() {
    throw "Значение";
}

int main() {
    try {
        f1();
    } catch (std::string &s) {
        std::cerr << "CATCH STRING";
        std::cerr << "(" << s << ")";
    } catch (int a) {
        std::cerr << "CATCH INT";
        std::cerr << "(" << a << ")";
    } catch (bool b) {
        std::cerr << "CATCH BOOL";
        std::cerr << "(" << b << ")";
    }

    return 0;
}
```

- g) Напишите фрагмент кода в котором выкидывается исключение, но никак не обрабатывается. Опишите наблюдаемое поведение.

```
#include <iostream>

void f1() {
    throw 42;
}

int main() {
    try {
        f1();
    } catch (std::string &s) {
        std::cerr << "CATCH STRING";
        std::cerr << "(" << s << ")";
    }

    return 0;
}
```

Наблюдаемое поведение : программа выдаёт ошибку и не заканчивает свою работу

h) Можно перехватывать исключения любого типа используя такой синтаксис:

```
#include <iostream>

void f1() {
    throw std::string("HELLO");
}

int main() {
    try {
        f1();
    } catch (...) {
        std::cerr << "CATCH STRING";
    }

    return 0;
}
```

i) Перехватите исключение out_of_range при помощи logic_error.

```
#include <iostream>
#include <vector>

int f(std::vector<int> &a, int n) {
    if (n > std::size(a)) {
        throw std::logic_error ("Out of range");
    }

    return a.at(n - 1);
}

int main() {
    int n;
    std::cin >> n;

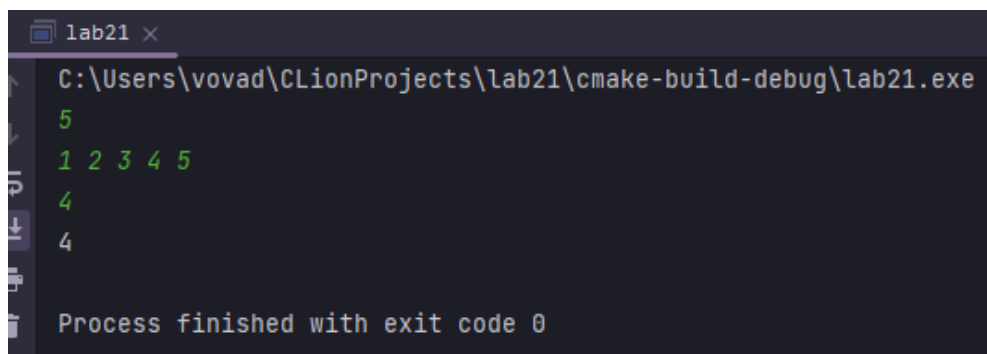
    std::vector<int> a(n);
    for (auto &x : a) {
        std::cin >> x;
    }

    int number;
    std::cin >> number;

    try {
        int res = f(a, number);
        std::cout << res << std::endl;
    } catch (const std::exception &e) {
        std::cerr << e.what();
    }

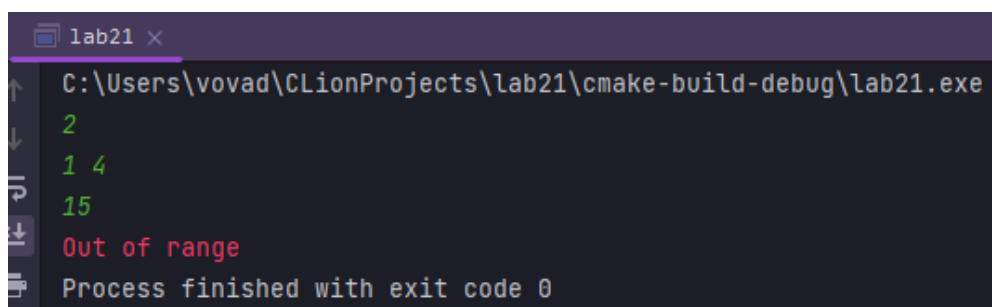
    return 0;
}
```

Вывод, не выходя за пределы вектора :



```
lab21 x
C:\Users\vovad\CLionProjects\lab21\cmake-build-debug\lab21.exe
5
1 2 3 4 5
4
4
Process finished with exit code 0
```

Вывод, выходя за пределы вектора :



```
lab21 x
C:\Users\vovad\CLionProjects\lab21\cmake-build-debug\lab21.exe
2
1 4
15
Out of range
Process finished with exit code 0
```

- j) Пусть есть некоторая функция, которая возвращает фрагмент свободной динамической памяти:

```
#include <iostream>
#include <vector>
#define BIG_NUMBER 100000000000LL

int* getmem(size_t partSize) {
    return new int[partSize];
}

std::vector<int*> getParts(int nParts, size_t partSize) {
    if (partSize >= BIG_NUMBER) {
        throw std::logic_error("Insufficient memory");
    }

    std::vector<int*> a(nParts);

    for (int i = 0; i < nParts; i++) {
        a.push_back(getmem(partSize));
    }

    return a;
}

int main() {
    int nParts, partSize;
    std::cin >> nParts >> partSize;

    try {
        std::vector<int*> a = getParts(nParts, partSize);

        for (auto x : a) {
            std::cout << x << ' ';
        }
    } catch (std::bad_alloc &e) {
        std::cout << e.what() << std::endl;
        return -1;
    }

    return 0;
}
```